AO * ALGORITHM

The start state and the target state are already known in the knowledge-based search strategy known as the **AO* algorithm**, and the best path is identified by heuristics

- Best-first search is what the AO* algorithm does. The AO* method divides any given difficult problem into a smaller group of problems that are then resolved using the AND-OR graph concept. AND OR graphs are specialized graphs that are used in problems that can be divided into smaller problems.
- The AND side of the graph represents a set of tasks that must be completed to achieve the main goal, while the OR side of the graph represents different methods for accomplishing the same main goal.

- The informed search technique considerably reduces the algorithm's time complexity. The AO* algorithm is far more effective in searching AND-OR trees than the A* algorithm.

Working of AO* algorithm:

The evaluation function in AO* looks like this:

f(n) = g(n) + h(n)

f(n) = Actual cost + Estimated cost

here,

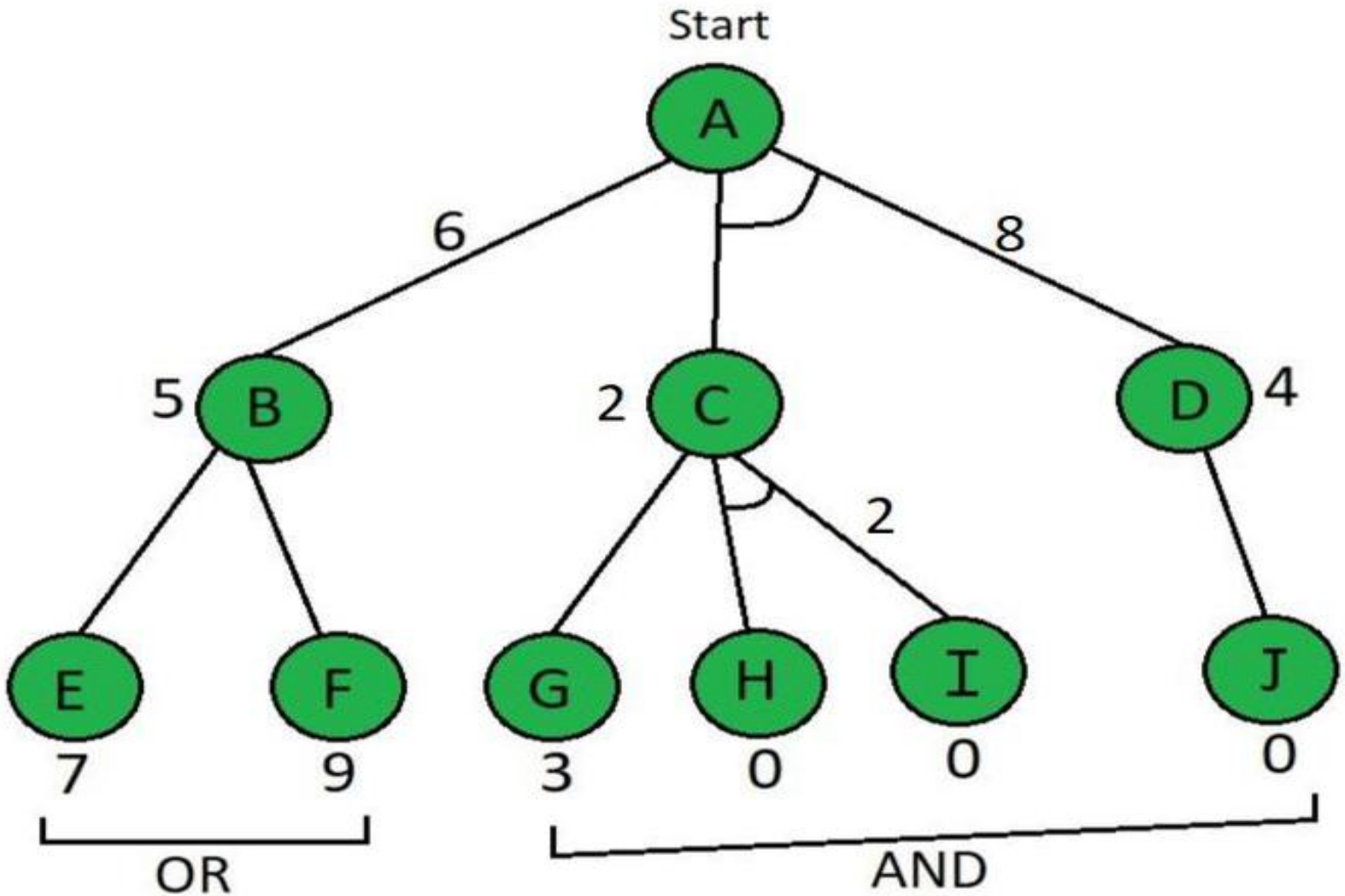
f(n) = The actual cost of traversal.

g(n) = the cost from the initial node to the current node.

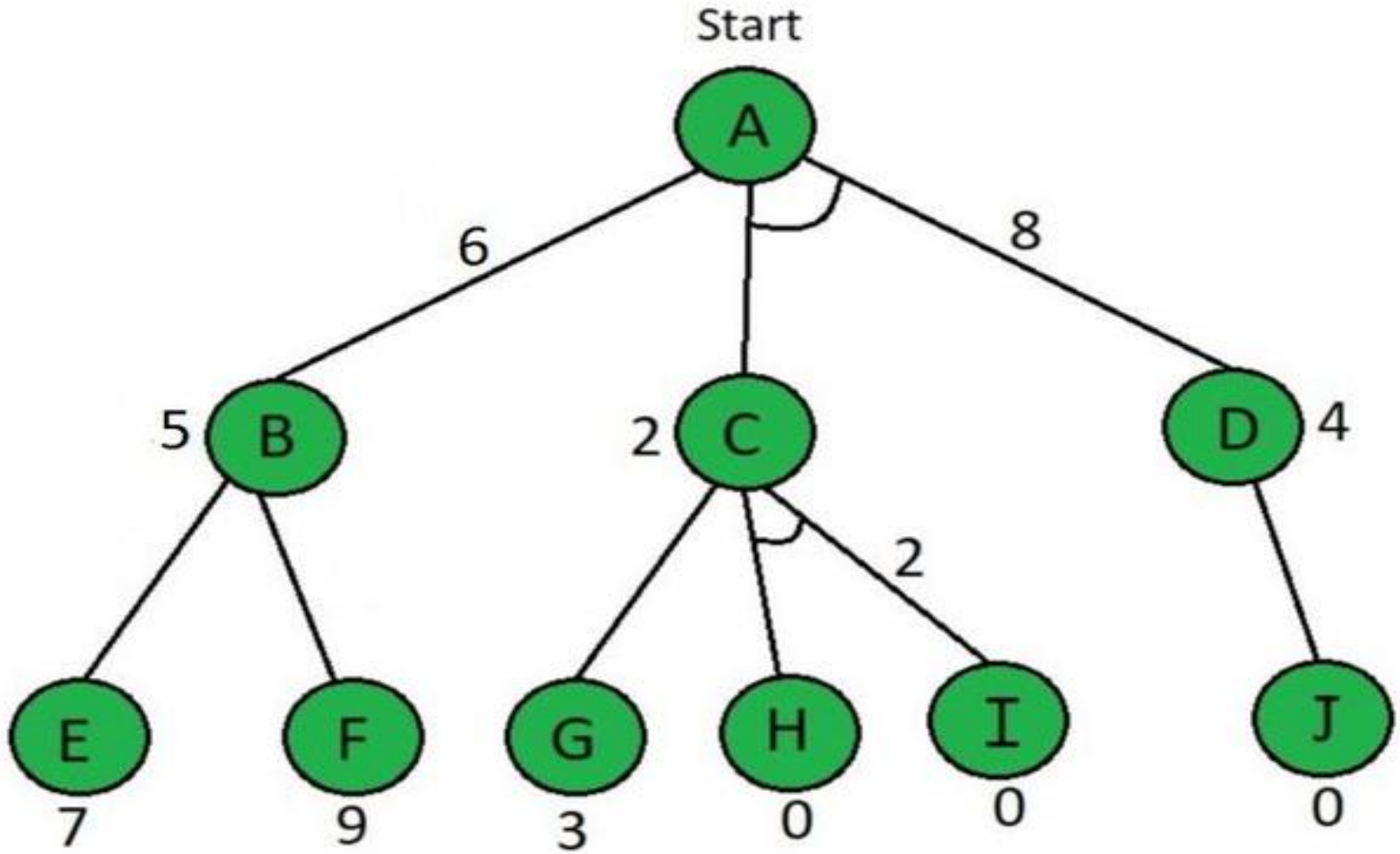
h(n) = estimated cost from the current node to the goal state.

Difference between the A* Algorithm and AO* algorithm

- A* algorithm and AO* algorithm both works on the best first search.
- They are both informed search and works on given heuristics values.
- A* always gives the optimal solution but AO* doesn't guarantee to give the optimal solution.
- Once AO* got a solution doesn't explore all possible paths but A* explores all paths.
- When compared to the A* algorithm, the AO* algorithm uses less memory.
- opposite to the A* algorithm, the AO* algorithm cannot go into an endless loop.


Start
A
6
8
5
B
2
C
D
4
2
E
F
G
H
I
J
7
9
3
0
0
0
OR
AND

- Here in the above example below the Node which is given is the heuristic value i.e h(n). Edge length is considered as 1.

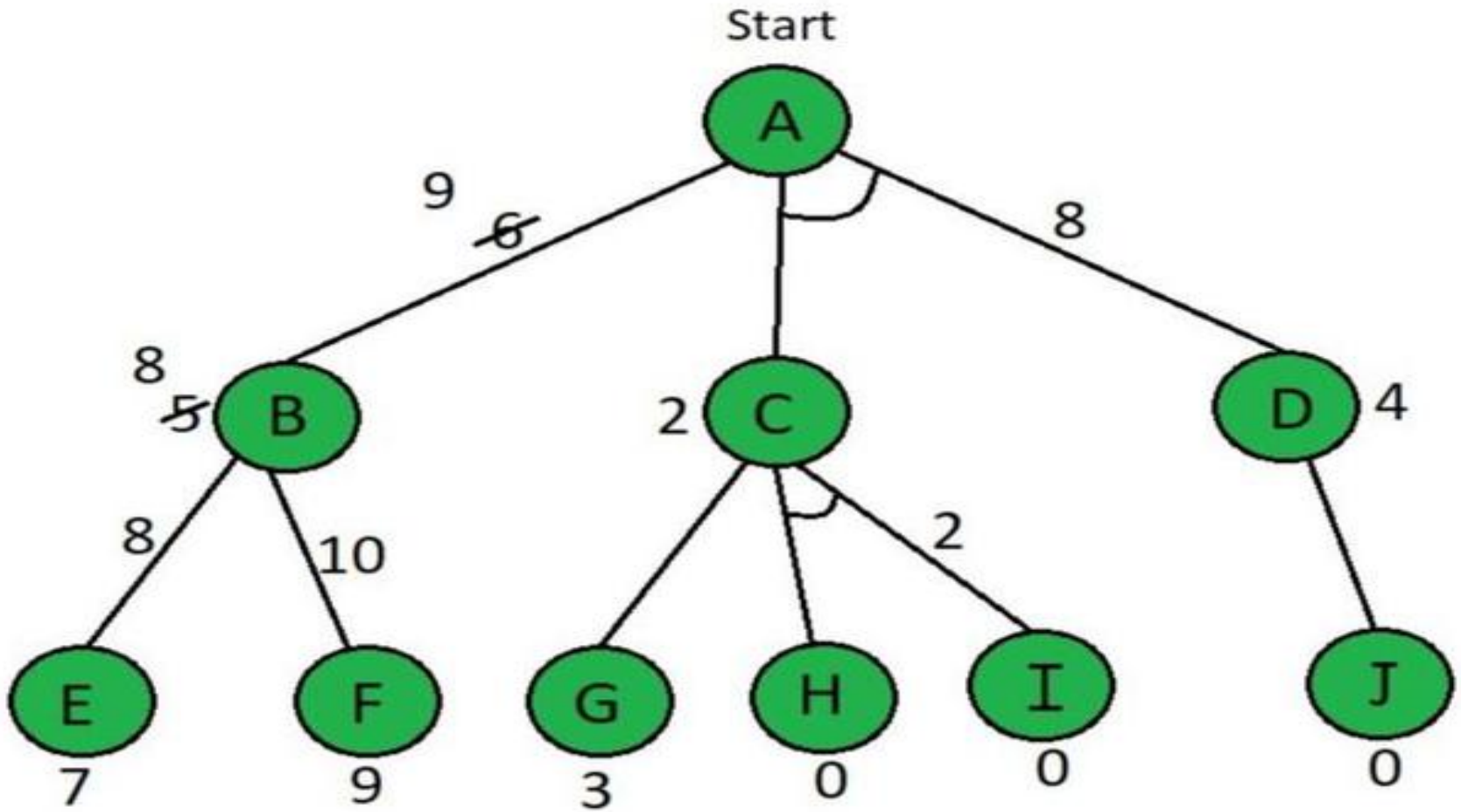

Start
A
6
8
5
B
2
C
D
4
2
E
F
G
H
I
J
7
9
3
0
0
0

Start from node A,
f(A→B) = g(B) + h(B) = 1 + 5 ……

here **g(n)=1** is taken by default for path cost = 6
f(A→C+D) = g(c) + h(c) + g(d) + h(d) = 1 + 2 + 1 + 4 ……

here we have added **C & D** because they are in **AND** = 8

So, by calculation **A→B** path is chosen which is the minimum path, i.e **f(A→B)**


Start
A
9
6
8
8
5
B
2
C
D
4
8
10
2
E
F
G
H
I
J
7
9
3
0
0
0

According to the answer of step 1,

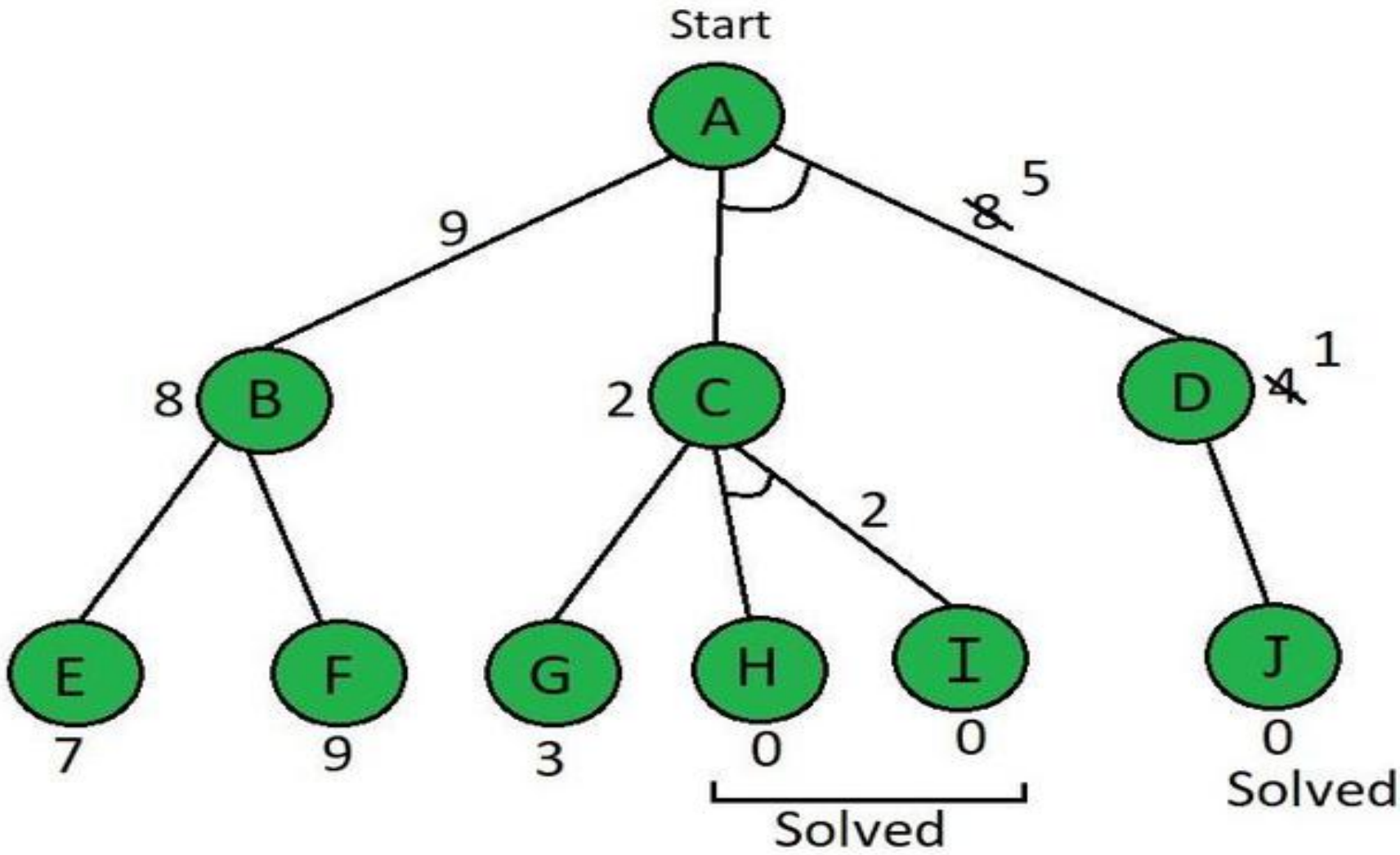
explore node B Here the value of E & F are calculated as follows,

f(B→E) = g(e) + h(e) f(B→E) = 1 + 7 = 8
f(B→f) = g(f) + h(f) f(B→f) = 1 + 9 = 10

So, by above calculation B→E path is chosen which is minimum path,
i.e **f(B→E)** because **B's** heuristic value is different from its actual value The heuristic is updated and the minimum cost path is selected.
The minimum value in our situation is **8**.

Therefore, the heuristic for **A** must be updated due to the change in **B's** heuristic.

So we need to calculate it again. f(A→B) = g(B) + updated h(B) = 1 + 8 = 9 We have Updated all values in the above tree.


Start
A
9
8
5
8
B
2
C
D
4
1
2
E
F
G
H
I
J
7
9
3
0
0
0
Solved
Solved

By comparing **f(A→B)** & **f(A→C+D)** f(A→C+D) is shown to be **smaller**.
i.e 8 < 9 Now explore f(A→C+D)

So, the current node is **C**

f(C→G) = g(g) + h(g) f(C→G) = 1 + 3 = 4

f(C→H+I) = g(h) + h(h) + g(i) + h(i)

f(C→H+I) = 1 + 0 + 1 + 0 ……here we have added **H & I** because they are in **AND** = 2

f(C→H+I) is selected as the path with the lowest cost and the heuristic is also left unchanged because it matches the actual cost.

Paths H & I are solved because the heuristic for those paths is **0,** but Path **A→D** needs to be calculated because it has an **AND**.

f(D→J) = g(j) + h(j) f(D→J) = 1 + 0 = 1 the heuristic of node D needs to be updated to 1.

f(A→C+D) = g(c) + h(c) + g(d) + h(d) = 1 + 2 + 1 + 1

- As we can see that path f(A⇢C+D) is get solved and this tree has become a solved tree now.

- In simple words, the main flow of this algorithm is that we have to find firstly level 1st heuristic value and then level 2nd and after that update the values with going upward means towards the root node